Infrastructure as Code

Service Orchestration in Dynamic Infrastructure

Ron C. Chiang

University of St. Thomas



Agenda

- Service discovery patterns
  - AWS Systems Manager Parameter Store
  - AWS Secrets Manager
  - AWS Cloud Map
- Container orchestration
  - AWS ECS
  - AWS Fargate

Service Discovery

- The process by which one service determines the location of another service.

- How do we connect services together in dynamically deployed infrastructure?
  - Example: Web service needs to know the location of the database service.

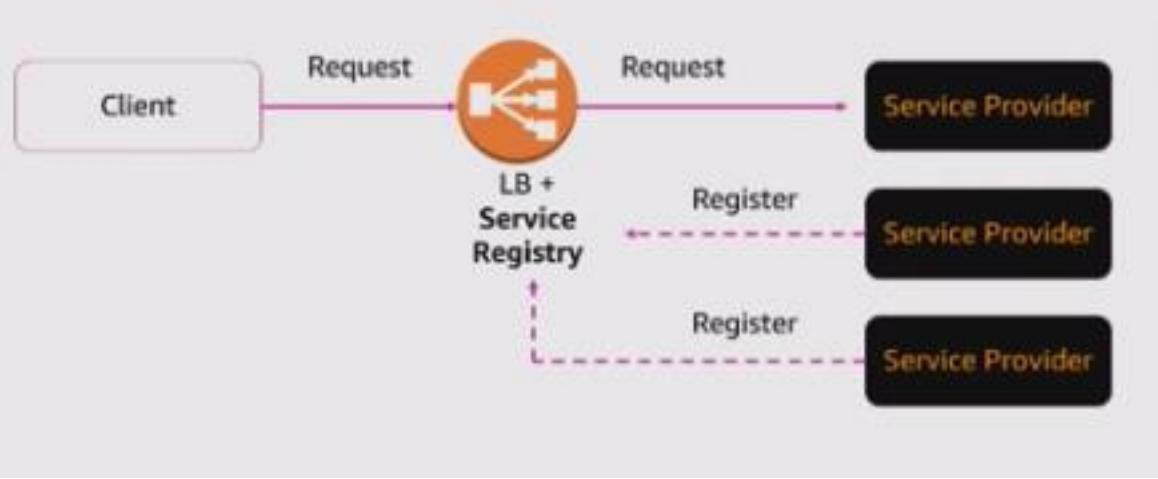
- This is one of the most critical challenges to solve in a distributed application architecture!

Service Discovery

- A distributed application platform heavily leverages service discovery:
  - Potentially needs to track dozens or hundreds of services.
  - May need to support service versioning.
  - Route service requests based on infrastructure topology changes.
    - Container or instance failure, auto-scaling

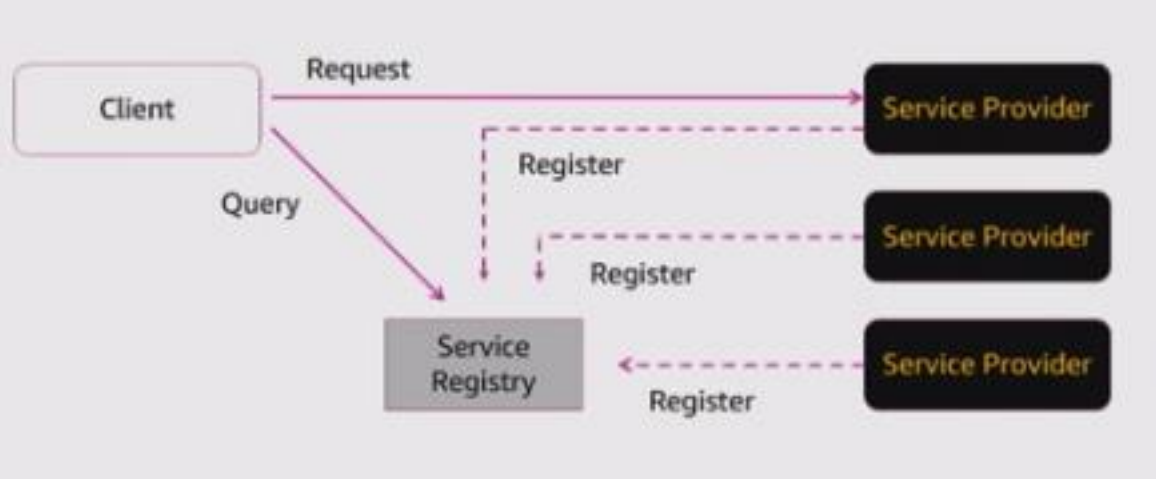
Service discovery patterns

- Two general patterns:
  - Server-side service discovery
    - Application connections are proxied and discovery is abstracted away.
    - Applications just need to know proxy endpoint.



Service discovery patterns

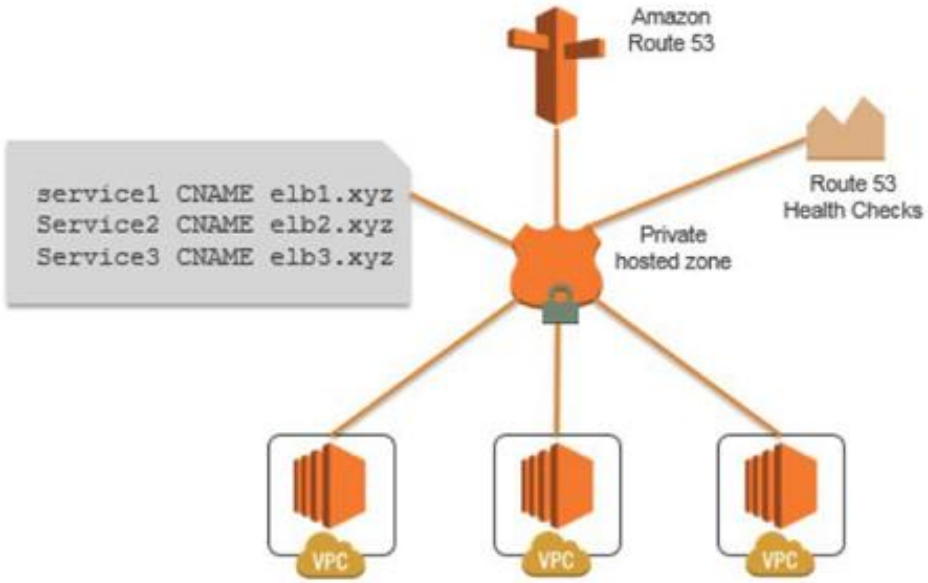
– Client-side service discovery
  - Applications connect directly to other services.
  - Clients communicate with a service registry.
  - Application clients handle load balancing.



Service discovery methods

- Traditionally we would hard-code service locations because infrastructure didn't change often.

- Domain Name Services
  - Discover the IP address of a system based on a name.
  - Not originally designed for dynamic infrastructure changes.



Service discovery methods

- Application load balancer service discovery
  - Abstracts the location for a single service
  - Doesn't address how a service finds the load balancer endpoint.



Source: http://www.slideshare.net/scaganoff/service-discovery-with-consul

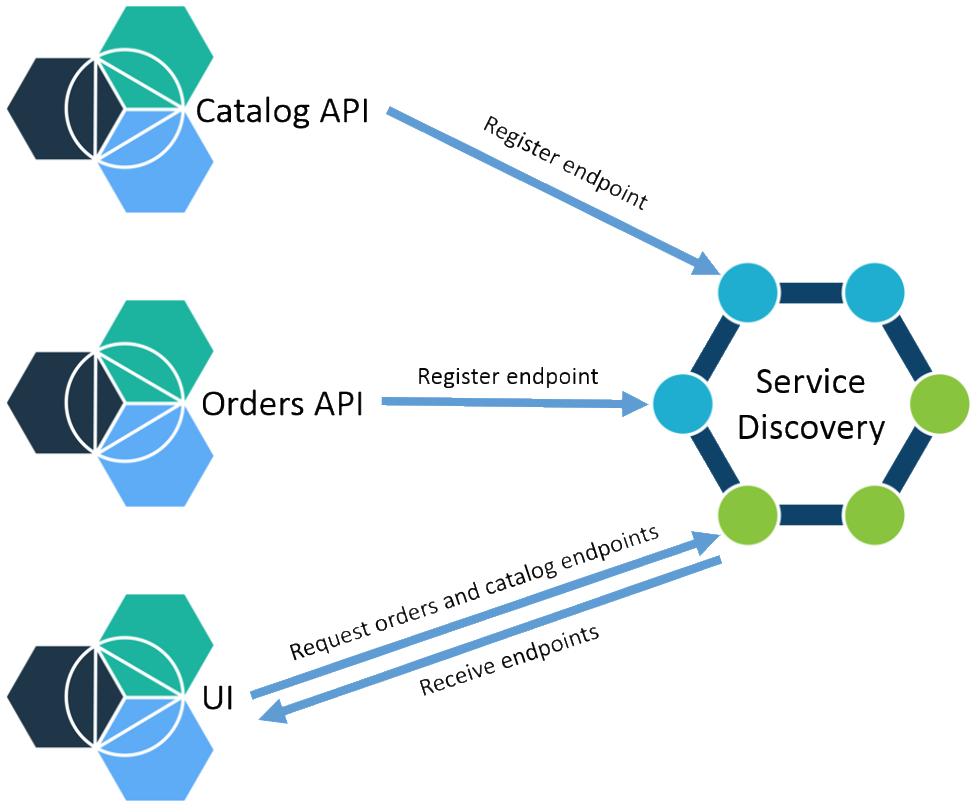
Service discovery methods

- Store service information in distributed file systems and databases
    - Distributed data storage challenges (lock servers)



Service discovery methods

- Dynamic service discovery platform



Service discovery solutions

- **Apache Zookeeper**
  - Originally started as a project at Yahoo, now a part of Apache Foundation.
  - https://zookeeper.apache.org/
- **Etcd**
  - Distributed consistent key-value store for shared configuration data.
  - Written in Go and maintained by CoreOS team.
    - https://github.com/coreos/etcd
  - Commonly used in container-based clusters (i.e., Kubernetes).
- **Consul**
  - Service discovery system designed by HashiCorp.
  - Distributed key/value store.

Source: https://en.wikipedia.org/wiki/Apache_ZooKeeper

AWS Systems Manager Parameter Store



- A key-value data store provided by AWS Systems Manager
  - Strings, StringList, SecureStrings
  - Standard: <10k parameters @ 4KB each (free)
  - Advanced: 10k+ parameters @ 8KB each ($)
  - Enable high throughput (1-1000/tps) ($)

AWS Systems Manager Parameter Store

```
# python get parameter example

import boto3
client = boto3.client('ssm')
Response = client.get_parameter(
        Name='myparameter',
        WithDecryption=False
)

# JSON response below

{
   "Parameter": {
      "Name": "myparameter",
      "LastModifiedDate": 1520896963.519,
      "Value": "secretpassword",
      "Version": 2,
      "Type": "String",
      "ARN": "arn:aws:ssm:us-east-1:419462506938:parameter/myparameter"
   }
}
```

AWS Secrets Manager



- A key-value data store for managing sensitive data
  - API tokens, service passwords, encryption keys
  - Can store encrypted strings like Parameter Store
  - Key values may be versioned
  - Integrated with RDS, Redshift, DocumentDB, and others
  - Supports automated key rotation using Lambda
  - $0.40/secret per month

AWS Secrets Manager

```
# python get secret example

import boto3
client = boto3.client(secretsmanager')
Response = client.get_secret_value(
        SecretId='company.financial.database',
        VersionId='1.1'
)

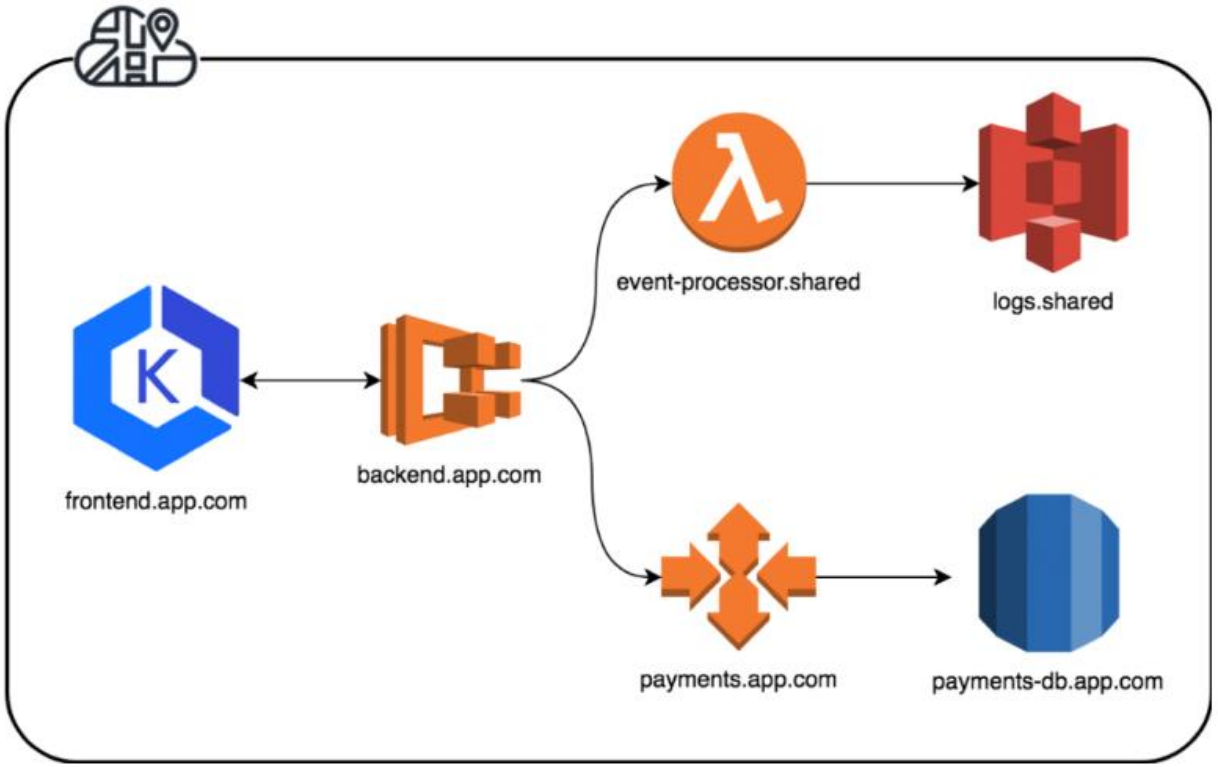
# JSON response below (truncated for brevity)

{
   "Name": "company.financial.database",
   "VersionId": "289f1f291-0b36-4ca8-9dec-sdfsdf2e12fg12g",
   "SecretString":
"{\"Certificate\":\"MIIJoQIsdf823hjsdf7881923rhf8y8127hf8912hvf9812rhjhgfv1regty8912737qGSIb3DQEHBq
CCA/WP+0epW+uvuFm1199DqTwpkwaEQQIgiiFtSgjnioCAggA\",\"Passphrase\":\"skjdf812jf9812hjf912ehfr\"}"
,
   "VersionStages": [
      "AWSCURRENT"
   ],
   "CreatedDate": 1529431126.997,
   "ARN": "arn:aws:secretsmanager:us-east-1:419462506938:secret: company.financial.database-VJ78GF"
}
```

AWS Cloud Map

- Managed service registry which tracks application components, locations, attributes, and health status.

- Allows applications to query Cloud Map using SDK, API, or DNS



AWS Cloud Map

- Define:
  - Namespace
    - logical name for grouping services
    - Configure instance discovery to use API or API + DNS
  - Service
    - Template for registering service instances
    - Specify IP routing (if using DNS)
    - Optionally define a health check
    - Define service instances
      - Attributes

AWS Cloud Map

- Discover instances using DNS or DiscoverInstances API

aws servicediscovery discover-instances --namespace-name myapp --service-name webservice


```
{
   "Instances": [
      {
         "InstanceId": "endpoint",
         "NamespaceName": "myapp",
         "ServiceName": "webservice",
         "HealthStatus": "UNKNOWN",
         "Attributes": {
            "url": "http://webservice.myapp.com/api/v1"
         }
      }
   ]
}
```

Orchestration

- A set of processes which control the lifecycle of infrastructure resources.
  - Like CRUD operations on compute, storage, and network resources.
  - AWS Console, EC2 Autoscaling
  - CloudFormation, Elastic Beanstalk, Ansible

Container orchestration

- Container orchestration needs to:
  - Schedule containers to run on computing instances.
  - Handle scenarios where a container or container host fails.
  - Distribute requests across sets of containers.
  - Deploy container image updates.
  - Provide a service discovery mechanism so services running in containers can find other services.

Container orchestration

- What container orchestration looks like:
  - Define containers and let the orchestration platform figure out the best location to run the containers.
  - The orchestration platform monitors state of containers and automatically handles topology changes.
  - The orchestration platform automatically sets up networking so that services in containers can communicate.

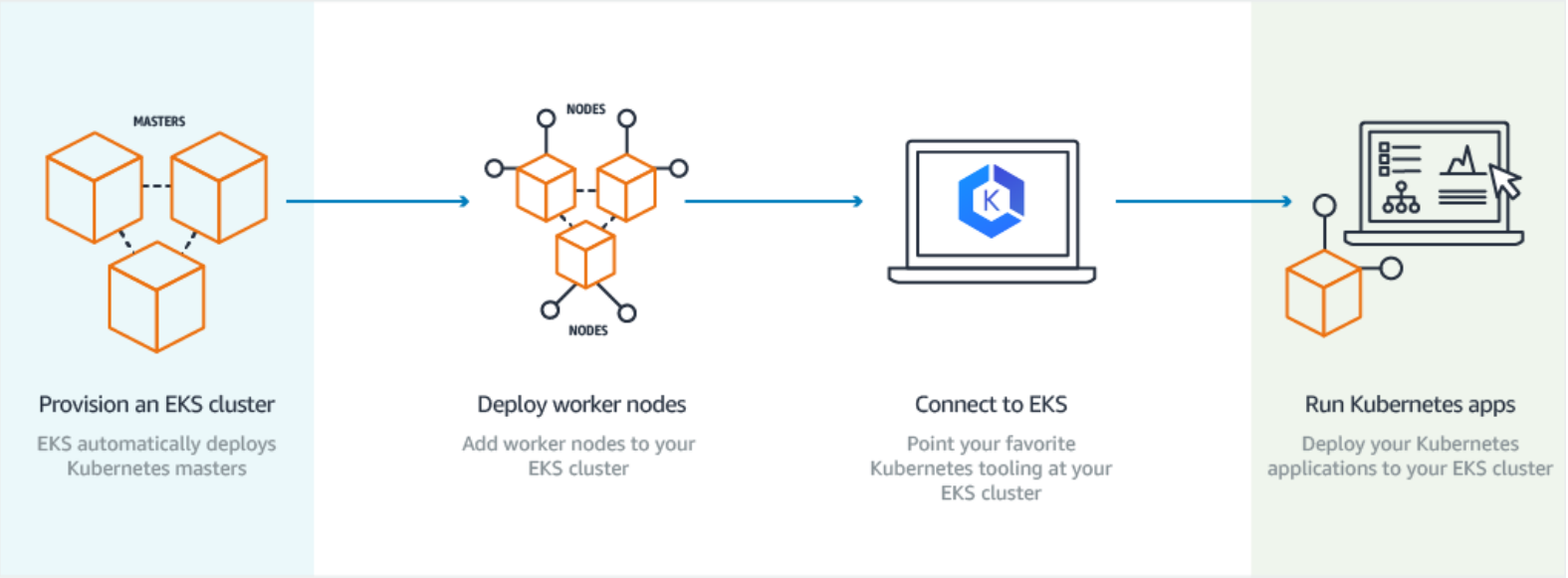
Kubernetes



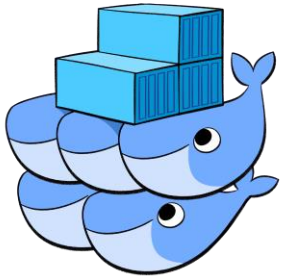
- Open-source container orchestration platform designed by Google.
- Name originates from Greek, meaning “helmsman” or “pilot”.
- Based on key technical principles Googles uses to scale its global platform.
- Each K8s cluster contains a master instance and multiple nodes, each running a Kubelet (agent) and engine (Docker).

AWS EKS

- Elastic Container Service for Kubernetes



Docker Swarm



- An integrated cluster orchestration service built into the Docker engine.
- Manages containers running on a cluster of **nodes** (EC2 instances).
- Uses two types of nodes: **managers** and **workers**.
  - Node type is defined during deployment.
- Follows a **declarative** approach: define the state of services in an application stack and Docker will try to achieve that desired state.

AWS Elastic Container Service (ECS)



- A Docker container management solution that runs on top of EC2 or Fargate.

- AWS deploys managed infrastructure to support your containers.
  - No need to build AMIs or configure servers.

- AWS automatically deploys your containers to the managed hosts and maintains a desired configuration state.
  - Define configuration in CloudFormation and JSON files.

- AWS EC2 Container Registry (ECR)
  - A private Docker image repository (like Docker Hub).

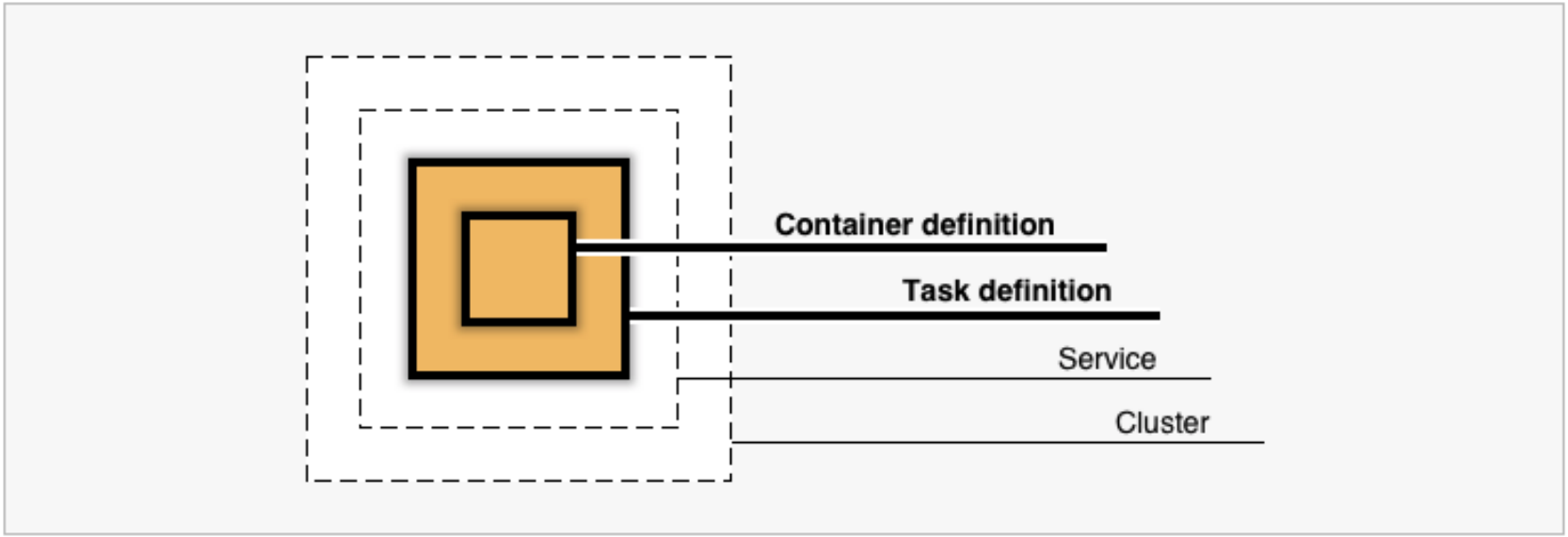
Fargate



- Run containers as a fundamental compute primitive.
  - We can launch and manage containers like EC2 instances.
  - Fully managed compute infrastructure: we have no access to the host servers.
- Default infrastructure for running containers in ECS.
- Integrates with other services – VPC, ELB, IAM, Cloudwatch, Cloudformation, etc.

Fargate components

Diagram of ECS objects and how they relate



Fargate components

- **Task definition**
  - A JSON file which describe how to configure containers (up to 10) that form your application.
  - Container image, exposed ports, data volumes, cpu/memory.

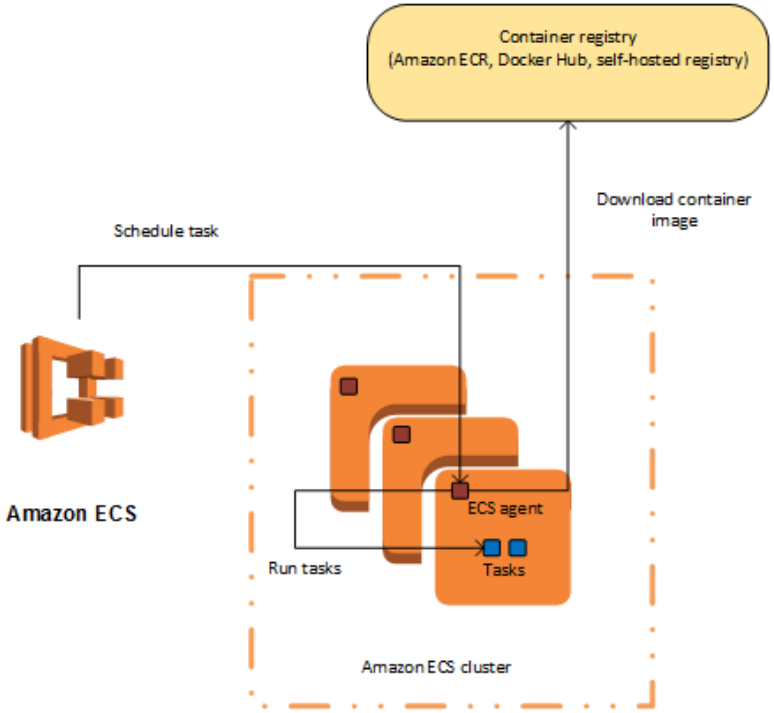
- A **task** represents the instantiation of a task definition in a computing cluster.
  - We can run multiple tasks in a cluster, each with it's own isolated compute and networking environment.
  - The ECS Task Scheduler is responsible for placing tasks within the cluster.

Fargate components

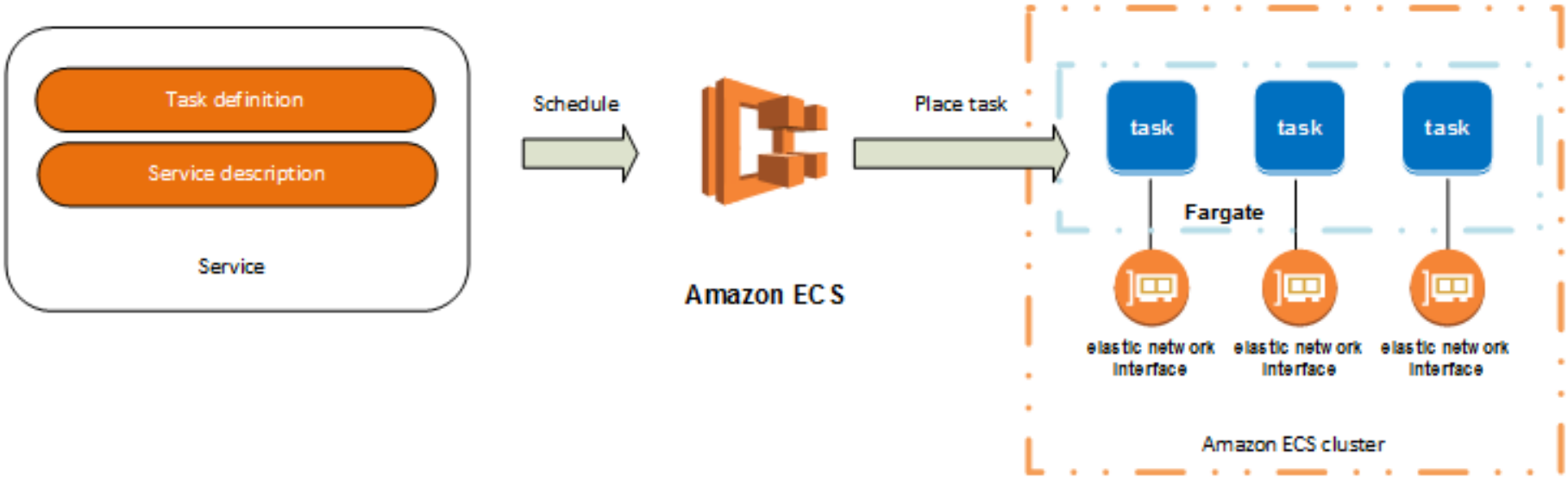
- **Service**
  - A service represents a specified number of task definitions running simultaneously in a cluster.
  - ECS will attempt to maintain a desired state despite task failures and topology changes.
  - Supports running multiple tasks behind an elastic load balancer for increased resiliency.
  - Integration with Route53 for service discovery.

Fargate components

- **Cluster**: a group of computing resources that host containers
  - Fully managed Fargate compute infrastructure
  - Can also use EC2 instances
  - Each compute node runs an ECS agent which manages tasks



Fargate components



Creating a cluster

- Three cluster options:
  - Networking only (Fargate)
  - EC2 Linux + Networking
  - EC2 Windows + Networking

- Cluster only has a name property

```
{    "Type" : "AWS::ECS::Cluster",
     "Properties" : { "ClusterName" : String }
}
```

Creating a service

- The service definition describes:
  - Type of hosting infrastructure (Fargate or EC2)
  - VPC subnet and security group
  - How many tasks to run simultaneously
  - How to deploy tasks
  - Load balancing
  - Service discovery

- Deployment types:
  - Rolling update
  - Blue/Green deployment (CodeDeploy)
  - External: custom deployment managed by API

Creating a service

- Load balancing
  - Containers that are launched in a single task definition are placed on the same host.
  - A service launching multiple tasks will deploy containers on separate hosts in multiple availability zones.
  - Use an application or network elastic load balancer to distribute requests across tasks.
    - A server can only have 1 associated load balancer.
    - Load balancer uses the task elastic IP in the target group

Creating a service

- Auto Scaling: ECS can automatically scale the number of tasks out or in
  - Specify desired number of tasks
  - Create a policy which scales tasks based on a CloudWatch metric (CPU%)

- Service discovery
  - ECS tightly integrated with AWS Cloud Map

Creating a service

```
"WebService": {
    "Type": "AWS::ECS::Service",
    "DependsOn": ["ELBListener"],
    "Properties": {
       "Cluster": {"Ref": "cluster"},
       "DesiredCount": "3",
       "LaunchType": "FARGATE",
       "TaskDefinition": { "Ref": "WebDefinition" },
       "NetworkConfiguration": {
          "AwsvpcConfiguration": {
             "AssignPublicIp": "ENABLED",
             "SecurityGroups": [{"Ref": "WebserverSG" }],
             "Subnets": [
                { "Ref": "PublicSubnet1"}                    ]
          }
       },
       "LoadBalancers": [
          {
             "ContainerName": "myContainer",
             "ContainerPort": "80",
             "TargetGroupArn": {"Ref": "ELBTargetGroup"}
          }
       ]

    }
 },
```

Creating a task definition

- Task definition parameters describe how to run containers:
  - Family: the name assigned to the task definition
  - Networking mode
    - Fargate only supports a networking mode called *awsvpc* where each task has a dedicated elastic network interface.
  - CPU/ memory to allocate to task
    - CPU increments: 256 (.25 vCPU) – 4096 (4 vCPU)
    - Memory increments: 0.5GB-30GB
  - IAM role tasks can use (like EC2 instance profile)
  - Container definitions: describes one or more containers to run in a task
    - Docker image to use
      - Pull from Docker Hub, ECR, or Private Docker repository

Creating a task definition

- CPU/ memory to allocate to container
- Logging configuration
  - Fargate uses the *awslogs* driver which sends logs to AWS CloudWatch Logs.
- Command container should run when it starts
- Task storage volumes
  - 10GB of ephemeral Docker layer storage
  - 4 GB of volume mount storage which may be shared

Creating a task definition

```
{
   "containerDefinitions": [
      {
         "command": [
            "/bin/sh -c \"echo '<html> <head> <title>Amazon ECS Sample App</title>
         ],
         "entryPoint": [
            "sh",
            "-c"
         ],
         "essential": true,
         "image": "httpd:2.4",
         "logConfiguration": {
            "logDriver": "awslogs",
            "options": {
               "awslogs-group" : "/ecs/fargate-task-definition",
               "awslogs-region": "us-east-1",
               "awslogs-stream-prefix": "ecs"
            }
         },
         "name": "sample-fargate-app",
         "portMappings": [
            {
               "containerPort": 80,
               "hostPort": 80,
               "protocol": "tcp"
            }
         ]
      }
   ],
   "cpu": "256",
   "executionRoleArn": "arn:aws:iam::012345678910:role/ecsTaskExecutionRole",
   "family": "fargate-task-definition",
   "memory": "512",
   "networkMode": "awsvpc",
   "requiresCompatibilities": [
       "FARGATE"
    ]
}
```

Architecting app on ECS

- Question: Multiple containers in the same task definition OR deploy containers in multiple task definitions?

- Single task definition:
  - Containers share common lifecycle
  - Containers reference each other on localhost port
  - Containers share resources or data volumes

- Separate task definitions:
  - Allow you to scale and manage containers separately

Task networking

- Fargate uses the *awsvpc* network mode
  - Every task gets an elastic network interface in a VPC
  - Similar to an EC2 instance
    - Primary private IP addresss and internal DNS hostname
    - May assign a public IP address
    - Needs to use a NAT gateway to access Internet from private subnet
    - Ability to associate security groups
    - Ability to use as a target for elastic load balancer

Secrets

- Tasks may need access to secrets like API tokens, passwords, and security keys.

- Inject sensitive data into containers using AWS Secrets Manager or Parameter Store.

```
"containerDefinitions": [
    {
        "secrets": [
            {
                "name": "environment_variable_name",
                "valueFrom": "arn:aws:ssm:region:aws_account_id:parameter/parameter_name"
            }
        ]
    }
]
```

Fargate Hands-on



- Launch a basic Fargate cluster:
  - https://seis665-public.s3.amazonaws.com/pets-basic-template.json
- Launch a more resilient Fargate cluster leveraging service discovery:
  - https://seis665-public.s3.amazonaws.com/pets-discovery-template.json
- Remember to delete the CloudFormation stack!